

PLATAFORMA DE TELEMEDICINA · MÓDULO 12 · VERSÃO 1.0

Videochamada da Consulta

O momento em que a telemedicina acontece. Tudo que os módulos anteriores construíram — agenda, pagamento, consentimento, fila, prontuário — existe para chegar aqui: duas pessoas, uma sala, e ninguém mais.

O que este módulo pressupõe

O Módulo 11 terminou: plantão e fila funcionando, `pnpm verificar` no [14/14].

O projeto é `telemedicina-modulo-12-videochamada.zip`. Testado do zero: **132 testes automáticos** (25 novos), tipos limpos nos 4 pacotes, 20 migrações, `next build` sem erro, 80 verificações contra o banco real (as 6 últimas sobre as salas de vídeo), e a API empacotada (a que roda no Render) conferida contra um banco criado do zero.

Os dois caminhos de banco foram testados: instalação nova e atualização a partir do Módulo 11. O `pg_dump` dos dois bancos é idêntico.

Este módulo corrige uma falha de segurança do Módulo 7

Enquanto escrevia a tela do paciente, encontrei um vazamento real: a rota `GET /consultas` devolvia a agenda **inteira** da clínica para qualquer pessoa logada — inclusive para um paciente, que recebia nome, horário e motivo da consulta de todos os outros pacientes.

Está corrigido, com teste de regressão. A seção 5 explica a causa, porque ela se repete em todo projeto que usa Supabase — e porque eu mesmo caí nela.

0. A pergunta antes do código: quem transporta o vídeo?

Duas pessoas em casa, cada uma atrás do roteador da sua operadora, querem se ver e se ouvir ao vivo. Isso parece simples e não é: na maior parte dos casos os dois computadores **não conseguem** se achar sozinhos na internet, porque o roteador de cada um esconde quem está atrás dele. Precisa de um servidor no meio para apresentar os dois (chamado TURN), e às vezes outro para redistribuir a imagem (chamado SFU).

Ter isso por conta própria significa máquina ligada 24 horas, custando dinheiro mesmo quando ninguém está em consulta, e virando problema nosso quando cair. E “cair”, aqui, quer dizer **consulta médica interrompida no meio**.

Por isso o vídeo roda num provedor. O escolhido foi o **Daily**, e ele fica atrás de uma interface nossa — trocar por Twilio, Vonage ou LiveKit é escrever um arquivo novo na mesma pasta, sem tocar nas telas. Mesma forma do pagamento (Módulo 10) e da assinatura (Módulo 9).

O que se ganha e o que se perde

Ganha: não operamos mídia em tempo real — que é uma especialidade inteira, não um detalhe de implementação.

Perde: dependemos de um terceiro para o ato central do produto. É uma troca consciente, registrada no ADR-0014.

E dá para rodar sem contratar nada?

Dá. O padrão do projeto é `VIDEO_PROVEDOR=local_teste`, que **não faz videochamada nenhuma**: a sala abre e, no lugar da imagem, aparece um aviso dizendo exatamente isso. Nenhuma câmera é ligada. Todo o resto do atendimento funciona — prontuário, receita, encerramento.

A API **se recusa a subir em produção** com ele, a menos que você declare `PERMITIR_VIDEO_SIMULADO=sim`. É o mesmo desenho da assinatura sem valor legal: o modo de teste tem que ser impossível de ligar sem querer no lugar errado.

Comando novo	O que faz
<code>pnpm db:verificar-video</code>	Procura o que não deveria existir : sala de consulta cancelada, duas salas para a mesma consulta, sala vencida sem encerramento, e coluna de token ou de gravação na tabela. É o passo [15/15].

1. Como a sala funciona

Três ideias sustentam o módulo. Vale ler devagar, porque cada uma evita um problema concreto.

1.1 Uma sala por consulta, criada quando alguém entra

A coluna `consulta_id` tem índice **único**. Uma consulta tem uma sala, sempre a mesma — se o médico recarrega a página ou o paciente cai e volta, os dois reencontram-se no mesmo lugar.

Sem essa trava, dois cliques no mesmo instante poderiam criar **duas** salas para a mesma consulta: cada pessoa numa sala vazia, esperando a outra. O banco impede isso em vez de confiar que o código lembre.

E a sala nasce na primeira entrada, não quando a consulta é marcada: consulta marcada é consulta que pode ser cancelada, remarcada ou esquecida — e sala criada no provedor ocupa cota e custa.

1.2 A porta não é o endereço, é o token

O endereço da sala **não é segredo**. Ele aparece na barra do navegador, vai para o histórico, pode ser colado num grupo sem pensar. Quem abre a porta é um **token de entrada**: de uma pessoa só, para uma consulta só, válido até o fim da janela.

E o token não é guardado em lugar nenhum

Token guardado em banco é credencial parada esperando vazamento. E pior: continuaria valendo depois de quem o pediu perder o direito de entrar.

Cada entrada emite um token novo. O que guardamos é só o endereço da sala, que por si não abre nada.

O passo `pnpm db:verificar-video` **falha** se alguém acrescentar uma coluna com “token” no nome nessa tabela.

A chave da API do provedor fica no servidor e **nunca** vai ao navegador: com ela se criaria sala na conta inteira. O navegador recebe só o token.

1.3 Duas pessoas, e o limite está no provedor

A sala é criada com `max_participants: 2`. Não é só o nosso código contando: é o provedor que hospeda recusando o terceiro. Mesmo que um token vaze, não há lugar para ninguém sentar.

Consequência aceita: hoje não dá para colocar intérprete de Libras, acompanhante ou residente na sala. Quando isso entrar, entra como participante *nomeado* na consulta — com registro de quem é e por que está ali — e não afrouxando o limite.

2. Quem entra na sala

Esta é a regra de acesso **mais estreita de toda a plataforma**, e a única que não olha o papel na clínica.

Em todas as outras tabelas, “a equipe da clínica vê” foi razoável: a recepção precisa da agenda, a administração precisa dos pagamentos. **Aqui não existe papel que sirva.** Uma consulta é entre duas pessoas, e só essas duas entram — nem a recepção, nem a administração, nem quem mantém a plataforma. Não é configurável.

A administração continua sabendo que o atendimento aconteceu (consultas, pagamentos, auditoria). O que ela não tem é caminho para dentro da sala.

O porteiro faz quatro perguntas, e todas têm que passar:

- 1 Esta consulta existe **nesta** clínica?
- 2 Você é o **paciente** ou o **médico** desta consulta?
- 3 A consulta está num estado que abre sala?
- 4 Estamos dentro da janela de horário dela?

Quem não é da consulta recebe **403**, com uma mensagem que não conta nada — nem nome, nem horário, nem motivo. Saber que aquela consulta existe já seria informação de saúde sobre outra pessoa.

3. A janela de horário (e por que ela existe)

Uma consulta é às 15h. Sem janela, o paciente poderia entrar na sala às 9h da manhã e ficar lá; o médico, na terça, entraria na sala da consulta de quinta. A janela é o que faz a sala existir **no horário do atendimento**, e não como um lugar permanente.

14:45	15:00	15:30	16:00
-----	-----	-----	
abre a sala (15 min antes)	início marcado	fim previsto	fecha (30 min depois)

- **15 minutos antes:** tempo de testar câmera e microfone com calma. É o que resolve a maior parte dos “não estou conseguindo entrar” antes de virar problema.
- **30 minutos depois do fim:** consulta atrasa. Sala que fecha na hora exata derrubaria o atendimento no meio.
- **O estado vem antes do relógio:** consulta cancelada, concluída ou sem pagamento não abre sala nem no horário certo.

A regra mora em um lugar só — e isso é de propósito

`packages/shared/src/video.ts` responde “pode entrar agora?”, e **duas** partes usam a mesma função: a API, para liberar o token, e a tela, para dizer “a sala abre às 14:45” em vez de mostrar um botão que responde erro.

Se cada lado tivesse sua cópia, um dia uma mudaria e a outra não — e o resultado seria a tela prometendo uma entrada que a API recusa.

A API continua sendo a palavra final. A tela só antecipa o que ela vai responder (o relógio do computador da pessoa pode estar errado).

4. Não há gravação

E não é esquecimento nem configuração que alguém pode ligar depois. A sala é criada **sem a capacidade** de gravar, e não existe coluna de gravação na tabela.

O motivo está no texto que o paciente leu, no Módulo 11:

Termo de telemedicina, versão 2026-09-01

“A videochamada não é gravada. O que fica registrado é o que o médico escrever no prontuário, como em qualquer consulta.”

Gravar consulta médica significaria guardar dado sensível de saúde em vídeo — com guarda, prazo de retenção, pedido de exclusão e vazamento possível. **Nada disso foi prometido ao paciente, e por isso nada disso existe.** Se um dia existir, será com consentimento próprio, separado, e versão nova do termo.

O **bate-papo escrito também fica desligado**, por um motivo parecido: o que é clinicamente relevante tem que ir para o prontuário, que fica registrado e assinado. Chat seria registro médico fora do registro médico.

4.1 O que fica registrado, sim

Entrada e encerramento vão para a auditoria. Para atendimento a distância, “quem esteve na sala e quando” é parte do que prova que o atendimento aconteceu — e a auditoria é imutável por gatilho desde o Módulo 4, inclusive para quem tem a chave secreta.

4.2 Encerrar apaga a sala no provedor

Quando o médico encerra, três coisas acontecem nesta ordem: a sala é **apagada no provedor**, o encerramento é marcado no banco, e a consulta vira concluída (e a linha da fila vira atendido).

Apagar no provedor é o que faz um token vazado deixar de valer. Sem isso, o resto seria só anotação.

Só o médico encerra. Não é hierarquia: encerrar fecha a sala para os dois, e o paciente podendo fazer isso derrubaria o médico no meio do atendimento — por engano, inclusive.

5. O erro que encontrei no Módulo 7 (e corriji aqui)

Esta seção é sobre uma falha minha. Ela vale mais que várias das explicações acima, porque a confusão que a causou se repete em quase todo projeto que usa Supabase.

5.1 O que estava acontecendo

A rota GET /consultas filtrava apenas por clínica. Resultado: **um paciente logado recebia a agenda inteira da clínica** — nome completo, horário e motivo da consulta de todos os outros pacientes. Dado de saúde de terceiro, entregue para quem não tinha nada a ver com aquilo.

A tela até dizia “Minhas consultas” para o paciente. A intenção estava certa desde o início; a API nunca a cumpriu.

5.2 A causa: uma linha de comentário errada

No lugar do filtro havia um comentário meu dizendo “o RLS já limita ao que a pessoa pode ver”. Não limita. Eu acreditei no próprio comentário e não conferi.

Entenda esta parte — ela decide a segurança de qualquer projeto com Supabase

O **RLS protege o acesso direto ao banco**: o navegador falando com o Postgres usando a chave publicável. Ali o Postgres sabe quem é a pessoa (`auth.uid()`) e aplica as políticas.

A API não entra por essa porta. Ela conecta com a credencial **dona** do banco — de propósito, para poder fazer o que o paciente não pode: criar consulta, mexer em pagamento, gravar auditoria. Para o Postgres, ela é o dono. **E dono não é limitado por RLS.**

Ou seja: **RLS é a segunda tranca, não a primeira.** Toda consulta feita pela API precisa filtrar por quem está pedindo. Sempre.

5.3 Como ficou

Quem pergunta	O que a API devolve agora
paciente	somente as consultas em que ele é o paciente
médico	as que ele atende (e as dele como paciente, se houver)
recepção / administração	a agenda da clínica — é o trabalho delas

E existe agora o **teste 13** em `apps/api/src/video.test.ts`, que cria dois pacientes de verdade no banco e falha se a consulta de um aparecer para o outro. Verifiquei que ele realmente pega o problema: tirei a correção, o teste quebrou; devolvi, passou.

Os outros lugares que poderiam ter o mesmo defeito foram conferidos um por um: `/documentos` e `/pacientes/:id/prontuario` já filtravam corretamente — o prontuário inclusive exige relação assistencial. O defeito era só na agenda.

6. Atualizar o projeto

Ordem obrigatória

1) `Ctrl+C` no `pnpm dev`. **2)** Extrair o zip por cima da pasta do projeto. **3)** `pnpm install` **antes** de qualquer outro comando — este módulo tem uma biblioteca nova.

```
pnpm install
pnpm typecheck
pnpm test
pnpm db:migrar
pnpm verificar
```

O que você deve ver

- `pnpm test`: **132 testes** (82 da API, 41 do shared, 5 do banco, 4 do site).
- `pnpm db:migrar`: *1 migracao(oes) aplicada(s), 20 de 20*.
- `pnpm verificar` no **[15/15]**, terminando em *Tudo certo*.

7. Usar

```
pnpm dev
```

Abra `http://localhost:3000`. Lembre que, sem conta no Daily, **não haverá imagem** — a própria sala avisa isso na tela. Todo o resto do caminho funciona, e é ele que vale conferir agora. A seção 8 ensina a ligar o vídeo de verdade.

7.1 Pelo pronto atendimento (o caminho mais curto)

- 1 Como **médico**, clique em **Plantão** na barra da clínica, depois em **Entrar de plantão agora** e em **Abrir plantão**.
- 2 Abra **outra janela do navegador** (uma anônima serve) e entre como **paciente**. Clique em **Pronto atendimento**, descreva o que sente, aceite o termo e pague — em desenvolvimento o pagamento é simulado, como no Módulo 10.
- 3 Você cai na **sala de espera**. Deixe esta janela aberta e visível.
- 4 Volte à janela do médico e clique em **Chamar próximo**.

- 5 **Olhe a janela do paciente.** Em poucos segundos ela vai sozinha para a sala da consulta — **não** mais para a agenda, como era no Módulo 11. Quem foi chamado cai onde foi chamado.
- 6 Na janela do médico você está na tela de atendimento, agora com o **vídeo na coluna da direita**, acima da emissão de documentos, e o prontuário à esquerda.

Repare: a câmera só liga depois do clique

Nenhuma das duas telas pede câmera ao carregar. O botão **Entrar na sala** é que começa tudo.

Isso é proposital: página que pede câmera sozinha faz o navegador mostrar o aviso de permissão no meio de outra coisa, a pessoa nega por reflexo — e aí não consegue mais entrar sem mexer em configuração do navegador. É onde o atendimento morre.

7.2 Clique em Entrar na sala, nas duas janelas

- Sem conta no Daily: aparece um painel explicando que este ambiente não faz videochamada, e dizendo qual variável configurar. **Nenhuma câmera foi aberta.**
- Com o Daily configurado: aparece a tela de testar câmera e microfone e, depois, a chamada. Repare na frase embaixo do vídeo: *“Esta chamada não é gravada.”*

7.3 Os testes que vale a pena fazer à mão

- **Recarregue** a página do paciente e entre de novo: cai na **mesma** sala. (É o índice único trabalhando.)
- Como médico, clique em **Encerrar atendimento**. A consulta vira concluída, o paciente sai da fila, e a sala **não abre mais** — tente entrar de novo pelas duas janelas e veja a recusa.
- Como **paciente**, procure um botão de encerrar: não existe. (A API também recusa, se alguém chamar a rota na mão.)
- Vá na agenda do paciente e olhe uma consulta **de outro dia**: em vez de botão, a tela diz a que horas a sala abre.

E o teste que mais importa, se você tiver um terceiro usuário na clínica: entre como ele (recepção, ou outro paciente) e tente abrir o endereço da consulta alheia, colando a URL. A resposta é recusa, sem revelar nada sobre a consulta.

7.4 Conferir no banco

```
-- as salas, e quem entrou nelas
select c.inicio, s.provedor, s.expira_em, s.encerrada_em
  from salas_de_video s join consultas c on c.id = s.consulta_id
 order by s.criada_em desc;

-- o registro de entrada e encerramento (imutável)
select acao, quando, detalhes from auditoria
 where acao like 'video.%' order by quando desc limit 10;
```

8. Ter videochamada de verdade (passo a passo)

São cinco minutos. O plano gratuito do Daily dá minutos de chamada por mês, suficiente para testar à vontade, e **não pede cartão**.

- 1 Abra <https://dashboard.daily.co> e crie sua conta (dá para entrar com Google ou GitHub).

- 2 Vai aparecer um pedido para escolher um **subdomínio** — algo como `minhaclinica.daily.co`. Escolha qualquer nome livre; você não vai precisar digitá-lo em lugar nenhum do projeto, porque o endereço da sala vem pronto na resposta do provedor.
- 3 No menu da **esquerda**, clique em **Developers**.
- 4 A primeira coisa da página é a **API key**. Clique no ícone de copiar ao lado dela.
- 5 Abra o arquivo `apps/api/.env` no seu editor. Vá até o fim e acrescente estas duas linhas, colando a chave depois do sinal de igual (sem espaços, sem aspas):

```
VIDEO_PROVEDOR=daily  
VIDEO_API_KEY=cole_aqui_a_chave_que_voce_copiou
```

- 6 Se já existir uma linha `VIDEO_PROVEDOR=local_teste` no arquivo, **apague** essa linha — a de baixo é a que vale, mas deixar as duas confunde quem ler depois.
- 7 Pare o `pnpm dev` (Ctrl+C) e rode de novo. **Variável de ambiente só é lida quando o servidor sobe.**
- 8 Entre na sala pelas duas janelas. Agora há câmera — autorize no navegador quando ele pedir.

Se der errado

A API não sobe e diz “VIDEO_PROVEDOR=daily exige VIDEO_API_KEY”: a linha da chave ficou vazia, ou foi salva em `apps/web/.env.local` por engano. Ela vai em `apps/api/.env`.

A tela diz que a chave foi recusada: a chave foi copiada pela metade. Volte em **Developers** e copie pelo botão, não selecionando com o mouse.

Aparece “não foi possível falar com o serviço de videochamada”: sem internet, ou o Daily fora do ar. A plataforma não quebra — ela avisa e deixa tentar de novo.

Para publicar com vídeo de verdade no Render, as mesmas duas variáveis entram em **Environment**, e a linha `PERMITIR_VIDEO_SIMULADO` deve ser **removida**. O guia `docs/publicar.md` já está atualizado.

9. O que cada arquivo novo faz

Arquivo	O que faz
packages/db/src/schema/salas-video.ts	A tabela salas_de_video: uma linha por consulta, com consulta_id único. Sem coluna de token e sem coluna de gravação — e o cabeçalho do arquivo explica por quê.
packages/db/drizzle/0019_sala_de_video.sql	A migração. No fim dela está a política de RLS mais estreita do banco: só o paciente e o médico daquela consulta.
packages/shared/src/video.ts	A regra da janela (decidirEntrada), usada pela API e pela tela. Uma fonte só.
apps/api/src/video/provedor.ts	O contrato que qualquer provedor de vídeo tem que cumprir. As quatro regras do módulo estão escritas aqui.
apps/api/src/video/provedor-daily.ts	A tradução para a API do Daily: criar sala privada, emitir token de uma pessoa, apagar a sala. É o único arquivo que fala HTTP com o provedor.
apps/api/src/video/provedor-local.ts	O dublê sem vídeo. Mesmo simulado, respeita as três coisas que os testes verificam: a sala tem validade, o token é de uma pessoa e expira, e apagar a sala invalida a entrada.
apps/api/src/video/resolvedor.ts	Decide qual provedor usar, uma vez , quando a API sobe — e recusa o simulado em produção sem autorização expressa.
apps/api/src/rotas/video.ts	O porteiro e as três rotas: entrar, olhar sem entrar, encerrar.
apps/web/src/componentes/SalaDeVideo.tsx	A sala na tela. Nada liga sem clique; a biblioteca do provedor só é baixada no clique (a tela do médico abre com 179 kB em vez de 248 kB); e sair da página desliga a câmera.
apps/web/.../consulta/[consulta]/page.tsx	A sala do paciente. Separada da do médico porque o que cada um faz na consulta é diferente — o médico escreve, o paciente conversa.
packages/db/src/verificar-video.ts	O passo [15/15]. Procura sala duplicada, sala de consulta cancelada, sala vencida sem encerramento, e coluna de token ou gravação.
apps/api/src/video.test.ts	14 testes contra o Postgres de verdade : cria clínica, médico e dois pacientes, e prova as regras de acesso. Inclui o teste 13, da falha da seção 5.
docs/adr-0014-...md	A decisão registrada, com as consequências aceitas e a correção de rota do Módulo 7.

10. O que eu não consegui testar

Como nos módulos anteriores, digo exatamente onde a prova para — é aí que você deve olhar com mais atenção.

- **O adaptador do Daily contra a API real.** Ele foi escrito a partir da documentação oficial dos endpoints de *rooms* e *meeting-tokens*, mas não tenho conta no Daily para bater contra o serviço. O que está provado é todo o resto: as regras de acesso, a janela, o encerramento e o comportamento das telas, com o provedor simulado.
- **As telas com sessão aberta no navegador.** Login depende do Supabase real, e as credenciais aqui são de teste. O que verifiquei: o `next build` passa, as duas rotas novas existem, e quem não está logado é mandado para o login e **devolvido à consulta depois** de entrar.

- **Duas pessoas em redes diferentes, de verdade.** Qualidade de imagem, queda de conexão e reconexão são do provedor, e só aparecem em uso real.

10.1 O que ficou pendente de propósito

- **Terceiro na sala** (intérprete de Libras, acompanhante, residente). Precisa ser participante nomeado, com registro — não é só aumentar o limite.
- **Sala de espera com aviso ao médico** quando o paciente chega antes. Hoje o paciente entra e espera na própria sala.
- **Reagendar quando ninguém apareceu.** A consulta fica *em andamento* até alguém encerrar; o passo [15/15] avisa quando isso se acumula.
- **A coluna de prioridade da triagem** (Módulo 11) continua sem tela que a use.

11. Git e checklist

```
git add -A
git commit -m "feat(video): sala de video por consulta, token por pessoa, sem gravacao"
git push
```

Se a plataforma estiver publicada, o push já atualiza a Vercel. No Render, acrescente as duas variáveis novas em **Environment** (VIDEO_PROVEDOR=local_teste e PERMITIR_VIDEO_SIMULADO=sim) — sem elas, a API **não sobe** em produção, e isso é o comportamento correto.

Checklist do Módulo 12

- [] pnpm install rodado **antes** dos outros comandos
- [] pnpm test: 132 testes passando
- [] pnpm db:migrar: 20 de 20
- [] pnpm verificar: [15/15] e *Tudo certo*
- [] Paciente chamado na fila cai **na sala**, não na agenda
- [] Recarregar a página devolve à **mesma** sala
- [] **Encerrar atendimento** conclui a consulta e a sala não reabre
- [] O paciente **não** tem botão de encerrar
- [] Consulta de outro dia mostra o horário de abertura, não um botão
- [] Variáveis novas acrescentadas no Render, se estiver publicado

Próximo módulo: validação de CRM e a ficha do profissional — conferir que quem diz ser médico está com registro ativo, que é o que falta para a plataforma poder abrir cadastro sem triagem manual.